

42. Un jeu lisible en couleur comme en nuances

Dessiner d'abord en quatre nuances, colorier ensuite : un jeu qui se lit toujours.

1. Ce qu'il faut comprendre

On choisit **une** console en haut de l'atelier : « **En couleur** » donne une cartouche `.gbc`, « **Game Boy** » un `.gb` en quatre nuances. Mais les couleurs ne changent rien aux **dessins** : une tuile garde ses indices de 0 à 3, et la palette seule dit quelle couleur montre chaque indice.

D'où la règle d'or : **dessine d'abord en quatre nuances**, avec des contrastes qui se lisent (chapitre 7), **puis** choisis les couleurs. Un rouge et un vert de même clarté se confondent pour qui voit mal les couleurs, sur un écran terne, ou en photo noir et blanc : un jeu qui ne se lit qu'en couleur est un jeu fragile.

Ici, le héros est dessiné avec un contour en 3, le sol avec une surface en 1 et 2 : il se détache du décor par la **clarté**, pas seulement par la couleur. En couleur, il est rouge sur un sol vert ; en nuances, sombre sur clair.

Pour vérifier **sans** changer de console : les **réglages** de l'atelier (⊗) ont une case « Voir en quatre nuances », qui montre la cartouche couleur telle qu'une Game Boy d'origine l'afficherait.

Une vraie **Game Boy Advance** lit aussi les cartouches `.gbc`, en couleur.

À toi : active « Voir en quatre nuances » et vérifie que ton héros se distingue toujours du sol.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **817 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          57 o ← la console, à l'allumage — la mise en route de la console
main()             283 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← ecran() — attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte — attend de pouvoir écrire une case
AdresseCase         10 o ← poser(), teindre() — trouve l'adresse d'une case de l'écran
EcrireTexte         10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins        11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles        85 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
LireManette         37 o ← bouton() — lit les huit boutons
Tuiles              208 o ← CopierTuiles — 11 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert    10 o ← InstallerTransfert — la petite routine qui copie les lutins
"GAUCHE DROITE"     13 o ← texte() — un texte gravé
```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```
couleurFond() × 4    48 o sur place, dans main()
couleurLutin() × 3   36 o sur place, dans main()
teindre()      × 1   24 o sur place, dans main() + AdresseCase (10 o)
```

```

sprite()      × 1    23 o sur place, dans main()
ecran()       × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
teindreLutin() × 1    16 o sur place, dans main()
poser()       × 1    13 o sur place, dans main() + AdresseCase (10 o)
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton()      × 2    10 o sur place, dans main() + LireManette (37 o)
image()       × 1     6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros rouge sur un sol vert, qu'on déplace avec GAUCHE / DROITE.
// Il reste lisible même sans couleurs : il est SOMBRE (contour en 3) sur un
// sol CLAIR (1 et 2). La règle d'or : dessiner d'abord en 4 nuances, puis colorer.

```

```

#include <Tuile>           // un dessin de 8 × 8 pixels
#include <couleurFond>     // choisit une couleur d'une palette du fond
#include <couleurLutin>    // choisit une couleur d'une palette des lutins
#include <ecran>           // éteint ou rallume l'écran
#include <texte>           // écrit un texte à l'écran
#include <poser>           // pose une tuile sur une case du fond
#include <teindre>         // met une case du fond dans une palette
#include <bouton>          // lit un bouton de la manette
#include <sprite>          // place un lutin de 8 × 8 au pixel près
#include <teindreLutin>    // met un lutin dans une palette

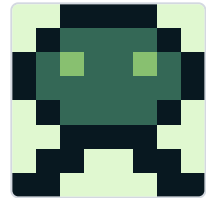
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {           // 0 transparent, 3 = le contour sombre.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "33000033",
};

```



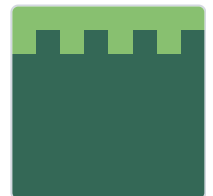
à l'écran

3. UN DESSIN — SOL

```

Tuile SOL = {             // Une surface claire (1), puis du 2.
    "11111111", "12121212", "22222222", "22222222",
    "22222222", "22222222", "22222222", "22222222",
};

```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    // La couleur : les palettes de la Game Boy Color
    // (sur une Game Boy d'origine, on verrait les 4 nuances à la place).
    couleurFond(0, 0, 20, 28, 31);          // le ciel      (palette 0, indice 0)
    couleurFond(0, 3, 2, 4, 12);            // le texte      (palette 0, indice 3)
    couleurFond(1, 1, 16, 30, 8);           // le sol        (palette 1, indice 1 : vert clair)
    couleurFond(1, 2, 4, 18, 2);           //                (palette 1, indice 2 : vert)
    couleurLutin(0, 1, 31, 28, 20);         // le héros      (palette 0 des lutins)
    couleurLutin(0, 2, 31, 4, 2);           //                rouge
    couleurLutin(0, 3, 10, 0, 0);           //                contour très sombre

    ecran(0);                               // Dessin écran éteint.
    texte(1, 1, "GAUCHE DROITE");
    for (uint8_t c = 0; c < 20; c++) {      // Toute la largeur,
        for (uint8_t l = 12; l < 18; l++) { // lignes 12 à 17 :
            poser(c, l, SOL);               // le sol,
            teindre(c, l, 1);               // en palette 1.
        }
    }
    ecran(1);                               // On rallume.

    uint8_t x = 76;                          // La place du héros, en pixels.

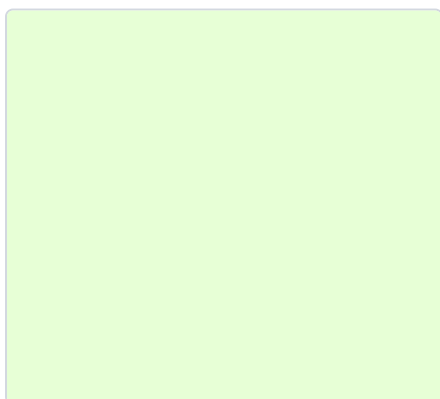
    while (true) {                          // La boucle du jeu :
        image();

        if (bouton(DROITE) && x < 152) x++; // à droite, jusqu'au pixel 152
        if (bouton(GAUCHE) && x > 8) x--;   // à gauche, jusqu'au pixel 8

        sprite(0, x, 88, HEROS);           // Le héros, posé juste au-dessus du sol (88 = 11 × 8).
        teindreLutin(0, 0);                // Après sprite() : sa palette (0).
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un héros rouge sur un sol vert, que GAUCHE et DROITE déplacent ; avec « Voir en quatre nuances », un héros sombre

sur un sol clair.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la cartouche est en couleur
- ✓ DROITE déplace le héros (76 puis 96)
- ✓ le héros est rouge (31/4/2)
- ✓ le sol est vert (4/18/2)

← [41. Animer les couleurs : l'eau qui scintille](#)

[sommaire](#)

[43. Au pixel près : le lutin glisse](#) →